

EnTraineR : construire des prompts pédagogiques à partir de sorties statistiques

Fiche synthétique pour comprendre le rôle du package dans un workflow d'interprétation

Enjeu

Dans un workflow statistique, les résultats sont souvent produits par des fonctions R, puis lus dans la console ou dans un rapport. Lorsque l'on souhaite utiliser un modèle de langage pour aider à l'interprétation, il ne suffit pas de lui poser une question générale.

L'enjeu est plutôt de construire un prompt contrôlé à partir de résultats statistiques explicites.

```
analyse statistique  
-> extraction des résultats  
-> construction du prompt  
-> contrôle humain  
-> génération optionnelle
```

Le package **EnTraineR** s'inscrit dans cette logique. Il aide à transformer des sorties statistiques en prompts pédagogiques, structurés et inspectables.

Rôle général du package

EnTraineR est conçu comme un assistant pédagogique pour l'interprétation de sorties statistiques en R.

Son rôle n'est pas de remplacer l'analyse statistique, mais d'organiser le passage entre :

- une sortie statistique explicite ;
- un contexte d'analyse ;
- des consignes d'interprétation ;
- un prompt prêt à être lu, vérifié ou envoyé à un modèle de langage.

L'idée centrale est donc :

```
sortie R explicite -> prompt pédagogique contrôlé
```

Ce que reçoit EnTraineR

Selon les fonctions utilisées, EnTraineR peut recevoir des objets ou sorties issus d'analyses statistiques classiques.

Exemples de sources possibles :

- une sortie de `FactoMineR::LinearModel()` ;
- une sortie de `FactoMineR::AovSum()` ;
- des tests statistiques classiques comme `t.test()`, `var.test()`, `prop.test()`, `cor.test()` ou `chisq.test()`.

Le point important est que le prompt doit s'appuyer sur des résultats déjà produits par R.

Ce que produit EnTraineR

La sortie principale d'EnTraineR est un prompt.

Ce prompt peut ensuite être :

- lu par l'enseignant ;
- discuté avec les étudiants ;
- copié dans un modèle de langage ;
- envoyé automatiquement à un moteur LLM si cette option est activée.

Dans un cadre pédagogique, le plus important est souvent le prompt lui-même, car il rend visible la manière dont la sortie statistique est transformée en consigne d'interprétation.

Pourquoi `generate = FALSE` est important

Dans un usage pédagogique, l'option `generate = FALSE` est essentielle.

Elle signifie que l'on souhaite construire le prompt sans appeler immédiatement le modèle de langage.

Cela permet de vérifier :

- quelles sorties statistiques sont transmises ;
- quelles consignes sont données ;
- quel niveau de langage est demandé ;
- quelles limites d'interprétation sont imposées ;
- si le prompt est adapté à l'objectif de cours.

Autrement dit, `generate = FALSE` maintient le contrôle humain avant toute génération.

Un workflow en cinq étapes

Le workflow typique peut être résumé ainsi.

Étape	Exemple
1. Analyse statistique	<code>LinearModel()</code> , <code>AovSum()</code> , <code>cor.test()</code> , <code>chisq.test()</code>
2. Extraction des résultats	F-test, T-test, coefficients, p-values, R2
3. Construction du prompt	contexte + résultats + consignes + format attendu
4. Contrôle humain	inspection du prompt avec <code>generate = FALSE</code>
5. Appel optionnel au LLM	génération avec un moteur et un modèle choisis

Ce workflow permet de ne pas demander directement au modèle de langage d'interpréter des données brutes.

Exemples de fonctions

Selon la version installée, EnTraineR peut proposer des fonctions destinées à différents types de sorties statistiques.

Exemples de fonctions ou familles de fonctions :

```
trainer_LinearModel()
trainer_AovSum()
trainer_t_test()
trainer_var_test()
trainer_prop_test()
trainer_cor_test()
trainer_chisq_test()
```

Ces fonctions ont pour point commun de construire un prompt à partir d'une sortie statistique identifiable.

Exemple : modèle linéaire

Pour un modèle linéaire produit avec `FactoMineR::LinearModel()`, l'objectif est de guider l'interprétation de la relation entre une variable à expliquer et une ou plusieurs variables explicatives.

Le prompt peut demander de distinguer :

- l'effet global du modèle ;
- les coefficients ;
- les signes des effets ;
- les éléments statistiquement significatifs ;
- les limites de l'interprétation.

Cela évite de passer trop vite d'une association statistique à une conclusion causale.

Exemple : ANOVA avec `AovSum()`

Pour une ANOVA, EnTraineR peut aider à construire un prompt qui distingue :

- les effets principaux ;
- les interactions ;
- les tests globaux ;
- les coefficients ou contrastes ;
- les précautions liées aux comparaisons multiples ou aux interprétations post-hoc.

Le modèle de langage reçoit ainsi un cadre précis pour interpréter la sortie.

Adapter le niveau d'explication

Une caractéristique importante d'EnTraineR est la possibilité d'adapter le prompt au public visé.

On peut demander une interprétation pour un public :

- débutant ;
- appliqué ;
- avancé.

Le même résultat statistique peut donc être expliqué différemment selon l'objectif pédagogique.

Ne pas inventer les résultats

Le principe de prudence central est le suivant : le modèle de langage ne doit pas inventer des nombres ou des résultats absents de la sortie R.

Le prompt doit donc rappeler que l'interprétation doit s'appuyer uniquement sur les résultats fournis.

Cela protège le workflow contre une interprétation plausible, mais non fondée sur les sorties statistiques.

Place d'EnTraineR dans le workflow pédagogique

EnTraineR intervient après l'analyse statistique et avant l'interprétation générée.

Il ne remplace pas :

- le choix du modèle ;
- la vérification des hypothèses ;
- la lecture statistique ;
- le jugement de l'analyste.

Il aide à formuler un prompt à partir de résultats déjà produits et inspectables.

Relation avec NaileR

Dans le parcours pédagogique, EnTraineR et NaileR n'ont pas exactement le même rôle.

Package	Rôle principal
EnTraineR	Construire des prompts pour des analyses statistiques explicites
NaileR	Prolonger l'interprétation vers des sorties comme <code>condes()</code> , <code>catdes()</code> , les axes, les classes ou les textes

EnTraineR est donc particulièrement adapté aux premières analyses explicites : modèles linéaires, ANOVA et tests classiques.

NaileR prolonge ensuite cette logique vers des objets plus descriptifs, multidimensionnels ou textuels.

Message pédagogique

EnTraineR ne doit pas être présenté comme un générateur automatique d'interprétations.

Il doit plutôt être compris comme un outil d'organisation du passage entre une sortie statistique et un prompt contrôlé.

```
analyser
-> extraire
-> structurer
-> prompter
-> éventuellement générer
```

Le point central est la traçabilité : chaque étape reste visible et inspectable.

Résumé

À retenir :

- EnTraineR transforme des résultats statistiques explicites en prompts pédagogiques.
- Le prompt est un objet inspectable avant toute génération.
- L'option `generate = FALSE` permet de garder le contrôle.
- Le package aide à adapter l'interprétation au public visé.
- Le modèle de langage ne doit utiliser que les résultats fournis.
- EnTraineR prépare le passage entre les statistiques classiques et l'interprétation assistée par LLM.

Installation depuis GitHub

Le package peut être installé depuis GitHub avec :

```
install.packages("remotes")
remotes::install_github("Sebastien-Le/EnTraineR")
```

Puis chargé avec :

```
library(EnTraineR)
```

Diagnostic dans R

Dans une séance pédagogique, on peut vérifier rapidement si le package est installé et quelles fonctions sont disponibles.

```
entrainer_disponible <- requireNamespace("EnTraineR", quietly = TRUE)

if (entrainer_disponible) {
  utils::packageVersion("EnTraineR")
  grep("^trainer_", getNamespaceExports("EnTraineR"), value = TRUE)
}
```

Ce diagnostic permet de distinguer le support théorique de l'environnement R réellement disponible.